

CI/CD Pipeline

Detailed walkthrough of the GitHub Actions pipeline for the ipecho service

Carlos Pereyra

July 20, 2026

This document describes, step by step, the continuous integration and delivery pipeline used by the ipecho service. It covers every job, the tools involved, the secrets required, the image tagging strategy, and a real troubleshooting story from bringing the pipeline to green.

FIELD	VALUE
Project	lab-devops
Component	ipecho (IP echo web service)
Repository	github.com/pereyra-carlos/devops-lab
CI system	GitHub Actions (.github/workflows/ci.yaml)
Registry	Docker Hub - docker.io/sauay/ipecho
Triggers	push to main, and every pull request
Status	Passing

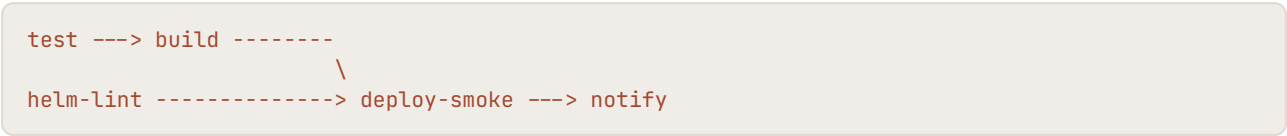
1. Overview

The pipeline runs on every push to the main branch and on every pull request. It is composed of five jobs that model a realistic path from source code to a verified deployment:

- test - static analysis and unit tests
- build - container build, vulnerability scan, SBOM, and push to the registry
- helm-lint - chart validation
- deploy-smoke - ephemeral Kubernetes deploy and post-deploy smoke test
- notify - success/failure notification

The design goal is that a red pipeline blocks a bad change early (tests, lint, or a critical CVE), while a green pipeline proves the image builds, scans clean, the chart is valid, and the application actually serves traffic once deployed.

Job dependency graph



test gates build. build and helm-lint both gate deploy-smoke. notify always runs last and reports the aggregate result.

JOB	PURPOSE	KEY TOOLS
test	Lint + unit tests	ruff, pytest
build	Build, scan, SBOM, push	docker, Trivy, actions/upload-artifact
helm-lint	Validate the Helm chart	helm lint
deploy-smoke	Deploy to kind + smoke test	kind, helm, kubectl, curl
notify	Report result	GitHub status, optional Slack

Stage by stage - what happens and why

1. Lint (ruff) - Static analysis without running the code: unused imports, dead variables, style. It is the fastest, cheapest check, so it runs first - if something this basic is wrong, the pipeline stops here before spending on a build.

2. Tests (pytest) - Runs the unit tests (IP resolution, geolocation, /stats, /ci, ...). A single failing test stops the pipeline. test is a dependency of everything else, so nothing proceeds if the logic is broken.

3. Trivy scan + SBOM - Trivy scans the freshly built image for known vulnerabilities (CVEs) in the base OS and dependencies; it is set to fail on CRITICAL (with ignore-unfixed, so it does not block on CVEs that have no patch yet). It also produces a CycloneDX SBOM - a full inventory of what is inside the image - saved as an artifact. This is the security gate.

4. Build (docker build) - Builds the multi-stage image: a builder stage that installs deps into a venv, and a minimal runtime stage (python:3.12-slim, non-root). Multi-stage keeps the final image small and free of the build toolchain - less attack surface, fewer CVEs.

5. Push to Docker Hub - Logs in with the DOCKERHUB secrets and pushes the image tagged with the commit SHA plus latest, only on push to main (never on pull requests) - so a fork PR can never push to the registry, since secrets are not shared with forks.

6. Helm lint - Validates that the chart (Chart.yaml, templates, values) renders correctly. Runs in parallel with build. Catches chart errors early - the missing ServiceAccount template bug (see the troubleshooting section) would have been caught here.

7. Ephemeral kind deploy - Spins up a throwaway kind (Kubernetes-in-Docker) cluster on the runner, loads the built image (pullPolicy: Never, no registry needed) and runs helm upgrade --install into a staging namespace. This exercises a real deployment, reproducibly, without touching any real infrastructure.

8. Smoke test - With the app running in kind, it port-forwards and curls /health and /ready. A non-200 fails the job. This is the final proof that the artifact actually starts and serves traffic.

9. Notify - Runs always (if: always()), whether the previous jobs passed or failed, and reports the aggregate result (optionally to Slack). It closes the loop.

Image tagging - Every image carries the exact commit SHA that produced it, and main also moves latest. This gives full traceability (you know exactly which commit runs in each image) and deterministic rollbacks (redeploying a SHA returns to an exact version).

2. Stage: test

The first job fails fast on style or logic errors before any image is built. It installs the runtime and dev dependencies, runs the ruff linter (a modern flake8 replacement), and then the pytest suite. Any lint error or failing test stops the whole pipeline here.

```
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: pip install -r app/requirements.txt -r app/requirements-dev.txt
    - name: Lint
      run: ruff check app
    - name: Unit tests
      working-directory: app
      run: pytest -q
```

3. Stage: build (build, scan, SBOM, push)

This is the security-critical job. It builds the container image tagged with the commit SHA, runs a Trivy vulnerability scan that FAILS the pipeline on any CRITICAL finding, generates a CycloneDX SBOM (Software Bill of Materials) as a build artifact, saves the image as an artifact for the downstream deploy job, and finally pushes to Docker Hub - but only on push events, not on pull requests (so forks/PRs cannot push).

Build and scan

```
- name: Build image
  run: docker build -t $IMAGE:${{ github.sha }} .
- name: Trivy vulnerability scan
  uses: aquasecurity/trivy-action@0.35.0
  with:
    image-ref: ${{ env.IMAGE }}:${{ github.sha }}
    format: table
    exit-code: "1"
    severity: CRITICAL
    ignore-unfixed: true
```

exit-code: 1 turns a CRITICAL finding into a pipeline failure. ignore-unfixed avoids failing on CVEs that have no fix available yet (which would otherwise block builds on issues you cannot action).

SBOM and image artifact

```
- name: Generate SBOM
  uses: aquasecurity/trivy-action@0.35.0
  with:
    scan-type: image
    image-ref: ${{ env.IMAGE }}:${{ github.sha }}
    format: cyclonedx
    output: sbom.cyclonedx.json
- uses: actions/upload-artifact@v4
  with:
    name: sbom
    path: sbom.cyclonedx.json
- name: Save image for downstream jobs
  run: docker save $IMAGE:${{ github.sha }} -o image.tar
- uses: actions/upload-artifact@v4
  with:
    name: image
    path: image.tar
```

The image is saved to a tar and uploaded as an artifact so the deploy-smoke job can load the exact same image into kind - even for pull requests that never push to the registry.

Push to Docker Hub

```
- name: Push to Docker Hub
  if: github.event_name == 'push'
  env:
    DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
    DOCKERHUB_TOKEN: ${ secrets.DOCKERHUB_TOKEN }
  run: |
    echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin
    docker tag $IMAGE:${{ github.sha }} $REGISTRY_IMAGE:${{ github.sha }}
    docker tag $IMAGE:${{ github.sha }} $REGISTRY_IMAGE:latest
    docker push $REGISTRY_IMAGE:${{ github.sha }}
    docker push $REGISTRY_IMAGE:latest
```

4. Stage: helm-lint

A lightweight job that runs helm lint against the chart. It runs in parallel with build and does not depend on it, so chart errors surface quickly.

```
helm-lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: azure/setup-helm@v4
    - run: helm lint charts/ipecho
```

5. Stage: deploy-smoke

This job proves the artifact actually runs. It spins up an ephemeral kind (Kubernetes-in-Docker) cluster on the runner, loads the image that build produced, installs the chart into a staging namespace with Helm, and then runs a smoke test with curl. If any probe never becomes ready or a smoke request fails, the job fails.

```
deploy-smoke:
  needs: [build, helm-lint]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with:
        name: image
    - uses: helm/kind-action@v1
      with:
        cluster_name: ci
    - name: Load image into kind
      run: kind load image-archive image.tar --name ci
    - uses: azure/setup-helm@v4
    - name: Deploy to staging
      run: |
        helm upgrade --install ipecho charts/ipecho \
          --namespace staging --create-namespace \
          --set image.repository=$IMAGE \
          --set image.tag=${{ github.sha }} \
          --set image.pullPolicy=Never \
          --wait --timeout 120s
    - name: Smoke test
      run: |
        kubectl -n staging port-forward svc/ipecho 8080:80 &
        sleep 5
        curl -f http://127.0.0.1:8080/health
        curl -f http://127.0.0.1:8080/ready
```

pullPolicy=Never forces Kubernetes to use the locally loaded image instead of pulling from a registry, which keeps the smoke test self-contained and works even on pull requests.

6. Stage: notify

The final job runs with `if: always()` so it reports whether the previous jobs succeeded or failed. A Slack step is included but gated behind a repository variable, so it is a no-op unless `SLACK_ENABLED` is set to true.

```
notify:
  needs: [test, build, helm-lint, deploy-smoke]
  if: always()
  runs-on: ubuntu-latest
  steps:
    - name: Report result
      run: echo "deploy-smoke: ${ needs.deploy-smoke.result }"
```

7. Secrets and configuration

The pipeline uses repository-level secrets (not environment secrets). No credential is ever hardcoded in the workflow or the chart.

NAME	TYPE	USED FOR
DOCKERHUB_USERNAME	Secret	Docker Hub login (value: sauay)
DOCKERHUB_TOKEN	Secret	Docker Hub access token (Read & Write)
SLACK_WEBHOOK_URL	Secret	Optional Slack notification
SLACK_ENABLED	Variable	Gate that enables the Slack step

8. Image tagging strategy

Every image is tagged with the full Git commit SHA, and main additionally moves the latest tag. This makes each build traceable to an exact commit and makes rollbacks deterministic.

```
docker.io/sauay/ipecho:<full-commit-sha> # immutable, one per commit
docker.io/sauay/ipecho:latest             # moves on every push to main
```

9. Troubleshooting story: bringing the pipeline to green

The pipeline did not pass on the first attempt. This section documents the real diagnosis and fix, because reading a failing pipeline is a core CI/CD skill.

Run #1 - failure

The first push produced a failed run. Inspecting the jobs showed test and helm-lint green, but the build job failed at the very first internal step, Set up job, before any of our commands ran. That is a strong signal the problem is the workflow definition itself, not our code.

The GitHub check annotations gave the exact cause:

```
Unable to resolve action aquasecurity/trivy-action@0.28.0,  
unable to find version 0.28.0
```

Root cause

The Trivy action was pinned to a tag (0.28.0) that does not exist in the aquasecurity/trivy-action repository. GitHub Actions resolves all referenced actions during Set up job, so an invalid version fails the job before execution. The valid recent releases are, for example, 0.35.0 and v0.36.0.

Fix and Run #2 - success

Both Trivy steps were re-pinned to a valid release (0.35.0) and pushed. Run #2 passed end to end: test, build (scan clean, SBOM produced, image pushed), helm-lint, deploy-smoke, and notify all green.

A secondary lesson: the very first run also lacked the DOCKERHUB_* secrets. Those were added as repository secrets before the fix, so by Run #2 the push step authenticated correctly.

Result: the pipeline runs lint, tests, a Trivy scan with an SBOM, builds and pushes a SHA-tagged image, validates the chart, deploys to an ephemeral cluster, and smoke-tests the running app on every push and pull request.

10. A note on delivery (CD)

The `deploy-smoke` job deploys to an ephemeral kind cluster that is destroyed when the run ends - it verifies the release, it is not the production target. The long-lived K3s cluster is deployed separately (via Helm). This is why the version running on K3s can lag the latest CI image: they are decoupled. The natural next step is GitOps (for example Argo CD) so the cluster continuously reconciles to the repository state.

lab-devops / ipecho - CI/CD Pipeline